ResilienceGuard

Evaluation Report / Synthetic experiment

Fixed seed 20261026 · reference 2026-10-26T08:00:00Z · no customer or production data

What was evaluated

Automated tests and a repeatable synthetic evaluation exercise whether backup files match their saved fingerprints, how old they are, whether service requirements are missing, and whether an approved sample file can be copied into an isolated folder. Three actual local copies were timed; those measurements cover file copying and sample-file checks only, not restoration of an application or database.

How to read the results: each fraction is the number of synthetic examples classified correctly divided by the number checked. For example, 5/5 means five of five examples matched their expected label. It does not mean five real services are safe or that real-world recovery is 100% reliable.

Automated suite	Passed	Failed	Total
pytest backend/tests	10	0	10

Measured checks

Check	Correct / total	Observed
File fingerprint: expected match / mismatch	5 / 5	100.0%
Backup age compared with data-loss target	5 / 5	100.0%
Missing backup inventory classification	4 / 4	100.0%
Service blockers across 3 scenarios	12 / 12	100.0%
Approved test copy, fingerprint, and file check	3 / 3	100.0%
Pending or untrusted copy was denied	2 / 2	100.0%
Assistant references matched allowed evidence and rules	1 / 1	100.0%

Restore timing and AI baseline

Approved file restore samples: 3. Mean observed time: 0.581 ms; minimum: 0.551 ms; maximum: 0.610 ms.

local file copy, source/destination SHA-256 checks, and synthetic JSON shape check; not application RTO

Rule-only baseline action IDs: clinical-db. Assistant mode: deterministic_fallback. Assistant action IDs: clinical-db. Plan action parity: True.

Evidence IDs and allowed action IDs were checked against the synthetic assessment: 1/1 (100.0%). The evaluated assistant used the deterministic fallback. not measured; no Qwen3/Ollama inference was run for this evaluation.

Security dependency scans

pip-audit: 0 known vulnerabilities found. npm audit: 0 findings. The PptxGenJS tree pins image-size 2.0.3 through an npm override after the initial scan found high-severity image-size parser advisories. The lockfile was re-audited after that fix. Results are point-in-time dependency scans, not a complete code/container penetration test.

The evaluation does not claim malware cleanliness, production recoverability, clinical application health, live PostgreSQL restore health, or generative-model accuracy.

Interpretation and limits

All fixtures are deterministic synthetic JSON with a fixed timestamp. Known expected labels are encoded in the evaluation script and matched against engine outputs. The sample size is deliberately small and tests correctness of the MVP's explicit rules, not performance at scale. The restoration test uses local filesystem copy operations; its measured milliseconds cannot be compared with a database or service RTO. The security evaluation is a dependency advisory check; it is not a full static/dynamic security assessment.

Unmeasured follow-up

- Run Qwen3 on a named hardware/runtime profile and score claims, evidence references, uncertainties and divergence from the rule-only baseline.
- Validate recovery plans with backup/vendor adapters and customer-owned failure-domain inventories.
- Exercise malware scanning, isolated application/database health checks, authentic identity/approval controls and independent audit protection.